

Pharmaceutical Document Change Analyzer Plan

Basic Application Flow

1. User uploads two versions of a document
2. Text is extracted from both documents
3. The filenames and extracted text are displayed to the user
4. The changes are identified through deterministic code or LLM call
5. Each change undergoes a risk classification process
6. The output summary is shown to the user, and can be edited, as well as downloaded for later viewing

Plan for Implementation

Text extraction (given in outline)

- PDF using PyMuPDF
- DOCX using python-docx
- TXT using normal file reading

Dividing the document into sections

Before we identify changes, it is important we find corresponding sections between the two documents. We cannot simply compare the two documents line by line, as paragraphs could have been moved around, deleted or modified extensively.

Method to identify sections

We cannot rely on the section number, position, or heading name in order to identify sections as these could change across versions. Hence, looking at the content of the section itself and assigning a similarity score might be a potential solution. We can then match sections with content that is similar to each other across two documents, making it easier for us to identify changes.

However, in the case that a section is inserted or deleted, it would mean that the paragraph would not match any section in the other document based on its similarity score.

In fact, instead of comparing only the body itself, we could compare the entire section which includes the heading. This could give us a more accurate similarity score between two sections even if headings change slightly.

Possible Methods to calculate similarity scores between sections

1. Jaccard Similarity

Each section is broken into a set of words and $\text{score} = (\text{words in both}) / (\text{words in either})$

This is a simple and fast implementation, but stop words need to be removed as they can inflate the score.

2. rapidfuzz

It is a python library which can be used to measure how similar two different strings are based on edit distance

`fuzz.ratio(a, b)` gives similarity over the whole strings.

“Sample Preparation” vs “Sample Preperation” would result in a high similarity score

3. TF-IDF +Cosine Similarity

1. Split each document into chunks

Break both documents into text chunks, giving a list of sections for v1 and a list for v2.

2.Vectorize

TF-IDF turns each section into a vector of numbers, one per vocabulary word, weighted by importance so distinctive words score high and common words score low. This vector represents the section.

3.Compare with cosine similarity

Cosine similarity measures the angle between two vectors, giving a score from 0 (nothing shared) to 1 (identical). Sections on the same topic share distinctive words, so their vectors point the same way and score high.

4.Match best pairs

For each old section, pick the new section with the highest cosine score, provided it clears a set threshold. If no new section clears it, that old section counts as deleted; any new section left unmatched counts as inserted.

TF-IDF + Cosine Similarity is not affected by number changes, heading changes, or positional changes. It looks at the distinctive words in a section.

However TF-IDF does not have a semantic understanding of words, hence if words are reworded, the similarity score between sections could drop, resulting in incorrect scores.

Due to this, we could use a transformer to convert sections into embeddings, and then calculate similarity scores, as these embeddings would also capture the meaning of each section.

Section Embeddings + Cosine Similarity Score

We load a pre trained model, and pass each chunk into the model to obtain its embeddings, and then we calculate cosine similarity of each embedding to find similar sections. It is similar to TF-IDF but instead of vectors we obtain embeddings for each section.

Final Choice

The method which I think should be used is the Section Embeddings + Cosine Similarity Score, as we would be able to capture the meaning of each section, helping us make correct alignments when matching corresponding sections.

Identifying Changes

Once we have matched the corresponding sections, we need to identify the changes.

For this, we can use the difflib library which would be able to detect the following:

- Added paragraph
- Deleted paragraph
- Modified paragraph
- Moved paragraph(requires further processing)

We can utilize regex filters on the output of the difflib sequence matching function to identify

- Numeric changes
- Unit changes
- Date changes

For the following changes, I feel that context of the entire document matters, and due to the various formats that these changes could have, identifying these changes deterministically might be difficult, so we could identify them using a simple llm call:

- Role or responsibility change
- Reference-document change
- Specification change
- Process sequence change

Classifying the Type of Change

Now that we have identified the changes, we need to classify them according to the following categories

For the following changes we can classify them directly using difflib:

- Added paragraph
- Deleted paragraph
- Modified paragraph
- Moved paragraph(requires further processing)

If any of the following changes is detected using one of the regex filters, we can then attach a classification label to that change:

- Numeric changes
- Unit changes
- Date changes

And for the remaining changes, as we pass them to an llm, the llm itself can provide us with the classification label as well:

- Role or responsibility change
- Reference-document change
- Specification change
- Process sequence change

Risk Classification

Now we have identified both the change, and its type, so we need to classify the risk of each change, based on the change type. For most changes, we follow a rule based approach

- Specification change: **High**
- Unit change: **High**
- Process sequence change: **High**
- Role or responsibility change: **Medium**
- Document reference change: **Medium**
- Test method reference change: **Medium**
- Additional clarification without meaning change: **Low** (can only be identified by an LLM)
- Formatting, spelling or punctuation: **Informational** (use an LLM/regex)

Displaying Reason for Risk Assessment

We should use an LLM to produce a short reason for each change, as the reason depends on the meaning, and context of the change, which an LLM is suited to handle.

Displaying Output

We have identified all of the changes, their category, risk, and reason, so we can produce output for the user to see

We can store all the information as a json field, and display a summary by counting the number of high, medium, and low-risk changes as well the total changes, and positional changes per section.

Furthermore, we display the table containing all the respective information per change, along with the option to filter based on risk, section, and change type

Whenever we find the changes between two documents we will create a json field, and its own comparison id. Each comparison field, and contain multiple change fields.

Lastly, the user can review and edit change information such as confirming risk classification, editing risk level, adding comments, and accepting changes. The user should also be able to download the report as CSV or JSON